

Why and When to Use Caching in Scalable Systems

Introduction

In today's performance-driven world of microservices and cloud-native applications, **caching is no longer optional**—it's a must-have for building **scalable**, **responsive**, and **cost-effective** systems.

As working with JVM-based applications, I've seen caching solve critical bottlenecks—but only **when used wisely**.

I'll walk you through:

- **When and why to use caching**
- **Redis vs in-memory cache (heap-based)**
- **Best practices for Java developers**
- X -> Pitfalls to avoid in production systems

Why Use Caching?

Every time your application hits a **database**, calls a **third-party API**, or performs **heavy computations**, it costs time and resources.

Caching answers:

- “How can I avoid making this expensive call again?”
- “Can I serve this faster if I already know the result?”

When to Use Caching

Use Case	Why Cache Helps
* Read-heavy endpoints	Reduces DB/API load
* Repetitive calculations	Speeds up response
* External API lookups	Saves latency & API call quota
* Product/category lists	Keeps response consistent & fast
Authentication/token checks Reduces overhead in auth services	

Redis vs JVM (Heap) Cache — Which One to Choose?

As Java developers, we often start with **in-memory caches** like Caffeine, Guava, or even simple ConcurrentHashMap. But here's the reality in production systems:

Feature	In-Memory Cache (Heap)	Redis Cache (External)
Shared across instances	✗ No	Yes (distributed)
Heap memory usage	High risk	✗ No JVM impact
GC pressure	Adds overhead	✗ Offloads from JVM
Scalability	✗ Limited to instance	Highly scalable
TTL / Expiry control	🚫 Manual	Built-in

Use in-memory cache for local quick lookups and Redis for shared, scalable caching.

Heap Memory Considerations:

In-memory caching directly consumes JVM heap, which can cause:

- ✗ OutOfMemoryError
- Increased GC pause times
- ! Latency spikes under load

Note: Always bound your local cache:

```
Cache<String, User> userCache = Caffeine.newBuilder()
    .maximumSize(5000)
    .expireAfterWrite(5, TimeUnit.MINUTES)
    .build();
```

And monitor with:

- jvisualvm, jconsole, Prometheus + Grafana

- Set proper -Xmx, -Xms, and GC strategies (G1GC preferred)

Redis Cache — My Go-To for Distributed, Safe Caching

Redis is:

- Blazing fast (in-memory DB)
- Supports TTL, eviction, persistence
- Easily integrates with Spring Boot (Lettuce, Jedis)

Example (Spring Boot + Redis):

```
@Cacheable(value = "userCache", key = "#userId")
public User getUserDetails(String userId) {
    return userRepository.findById(userId).orElse(null);
}
```

And configure `@EnableCaching` + Redis connection in `application.properties`.

Common Caching Pitfalls to Avoid

Pitfall	How to Avoid
Stale data ->	Use TTLs and cache invalidation
Unbounded heap cache ->	Always set max size + <code>expireAfter</code>
Cache stampede ->	Use distributed locks or bulk loads
Write inconsistency ->	Consider write-through caching

Final Thoughts

As systems grow, caching becomes your secret weapon — but only when balanced carefully with your memory budget and consistency needs.

As a Java engineer:

- Start with Caffeine for local, fast reads
- Adopt Redis for shared, cross-instance cache
- Monitor your heap health—cache should speed you up, not take you down

Remember: Caching is not about storing everything — it's about storing the right things smartly.